

PRONTUÁRIO: _____ **Data:** __/__/____ **Horário:** 3h00

✓ A prova é individual e sem consulta. ✓ Não é permitido utilizar qualquer código implementado por você anteriormente. ✓ Não coloque seu nome no projeto. ✓ Atribui-se nota zero à prova em desacordo com os itens acima.	✓ A prova deve ser nomeada da seguinte forma: PRONTUARIO_P_SIMULADA, com o "SC". ✓ Não envie apenas as classes, mas todo o projeto. ✓ Envie o projeto como zip (ou apenas os .java, se for só treino pessoal).
--	--

Considere o contexto a seguir:

Você foi contratado(a) para desenvolver o núcleo de um sistema de locadora de filmes. Você deve implementar as quatro classes do modelo — Filme, Cliente, Locacao (representa o aluguel de um filme por um cliente) e Locadora (gerencia todas as locações) — seguindo o diagrama UML a seguir:

Filme	Cliente
- titulo: String - genero: String - valorDiaria: double + Filme(titulo: String, genero: String, valorDiaria: double) + getTitulo(): String + getValorDiaria(): double	- nome: String - cpf: String + Cliente(nome: String, cpf: String) + getNome(): String + getCpf(): String
Locadora	Locacao
- locacoes: Locacao[] - proximoId: int + Locadora() + alugarFilme(cliente: Cliente, filme: Filme, dias: int): Locacao + devolverFilme(idLocacao: int): void + locacoesAtrasadas(): Locacao[] + totalArrecadado(): double	- id: int - cliente: Cliente - filme: Filme - dataRetirada: LocalDate - dataPrevistaDevolucao: LocalDate - dataDevolucaoReal: LocalDate + Locacao(id: int, cliente: Cliente, filme: Filme, dias: int) + registrarDevolucao(): void + estaAtrasada(): boolean + diasDeAtraso(): int + valorTotal(): double

Relacionamentos: Locadora possui de 0 a N objetos Locacao (composição, array). Cada Locacao está associada a exatamente 1 Cliente e 1 Filme (relacionamentos unidirecionais, setas de Locacao para Cliente e de Locacao para Filme — Locacao não tem nenhum objeto Locadora, Cliente não tem nenhum objeto Locacao, e assim por diante).

Regras de negócio:

- Ao alugar um filme, a diária do filme é multiplicada pela quantidade de dias solicitada para compor o valor total previsto da locação.
- dataRetirada é sempre a data do momento da criação da Locacao (use LocalDate.now()).
- dataPrevistaDevolucao é dataRetirada somada à quantidade de dias contratada (utilize a API de Date/Time — plusDays — não faça a soma manualmente).
- registrarDevolucao() define dataDevolucaoReal como a data atual (LocalDate.now()).

- Uma locação está atrasada se: (a) ainda não foi devolvida e a data atual já passou da dataPrevistaDevolucao; ou (b) já foi devolvida, mas dataDevolucaoReal é posterior à dataPrevistaDevolucao.
- diasDeAtraso() retorna a quantidade de dias de atraso (0 se não houver atraso), calculada usando a diferença entre a data de devolução (real, se já devolvido; ou a atual, se ainda em aberto) e a data prevista. Utilize ChronoUnit.DAYS.between ou equivalente da API Date/Time — não calcule manualmente a diferença de dias.
- valorTotal() é o valor previsto da locação (diária × dias contratados) acrescido de uma multa de 20% da diária por cada dia de atraso.
- locacoesAtrasadas(), em Locadora, retorna todas as locações (devolvidas ou não) que estão/estiveram em atraso.
- totalArrecadado() retorna a soma do valorTotal() de todas as locações já devolvidas.

Execute as atividades a seguir para a implementação do exercício em Java. Para a atribuição da nota será levada em conta não apenas a funcionalidade, mas a qualidade, adequação e pertinência de cada solução.

#	Descrição	Pont.
1	Implemente as classes Filme e Cliente com os atributos e métodos previstos no diagrama, com os modificadores de acesso adequados ao encapsulamento.	0,5pt
2	Crie as demais classes do modelo (Locadora e Locacao) com os modificadores de acesso adequados ao encapsulamento (atributos privados, métodos públicos onde fizer sentido).	0,5pt
3	Estruture a associação de composição entre Locadora e Locacao usando array.	0,5pt
4	Implemente o construtor de Locacao, calculando corretamente dataPrevistaDevolucao a partir de dataRetirada e dias, usando a API java.time.	1,5pt
5	Implemente o método registrarDevolucao() e o método estaAtrasada(), cobrindo os dois casos (em aberto e já devolvida).	1,5pt
6	Implemente o método diasDeAtraso() usando ChronoUnit (ou equivalente), retornando 0 quando não há atraso.	1,5pt
7	Implemente o método valorTotal(), aplicando corretamente a multa por atraso.	1,0pt
8	Implemente o método alugarFilme(...) da Locadora, criando e armazenando a Locacao no array (gerenciando o id automaticamente).	1,0pt
9	Implemente o método devolverFilme(idLocacao) da Locadora, localizando a locação pelo id e chamando registrarDevolucao().	1,0pt
10	Implemente locacoesAtrasadas() e totalArrecadado().	1,0pt
11	Crie uma classe Principal com um método main() que cadastre ao menos 2 clientes e 2 filmes, realize 3 locações (com pelo menos uma delas simulando atraso, alterando manualmente a data prevista para uma data passada), devolva-as, e imprima o total arrecadado e as locações atrasadas.	0,5pt

Observação: para simular atrasos em tempo de teste, você pode expor um método de apoio (ou setter interno) apenas para fins de teste na classe Principal, sem quebrar o encapsulamento das demais classes — deixe claro no código que esse recurso é só para simulação.